

Proyecto Final Informática II: Cyberpunk 2077

Ramón F. Santacruz M.

*Facultad de Ingeniería
Universidad de Antioquia*

FRANCISCO.SANTACRUZ@UDEA.EDU.CO

Juan David Naranjo Jara

*Facultad de Ingeniería
Universidad de Antioquia*

JUAN.NARANJOJ@UDEA.EDU.CO

1 Introducción

Se planteó como proyecto final del curso, realizar un videojuego, de tal manera que nosotros como estudiantes apliquemos todos los conceptos aprendidos durante el semestre para cumplir con la entrega. La temática rondaba acerca de algún deporte en un universo/contexto particular. Nosotros presentamos la idea de un videojuego basado en parkour en el universo de Cyberpunk 2077.

Kael, es un chico que lucha en contra de la inteligencia artificial, la cual a tomado el control del mundo, para ello, Kael deberá acceder a la red central del sistema y hackearlo, teniendo así que subir plataformas, luchar contra el viento, evadir los robots, esconderse y correr. Planteamos dos niveles, uno de plataformeo y otro pensado más en sigilo, disponiendo así de dos configuraciones visuales de los escenarios, ambientación sonora y un pixelart futurista.

2 Objetivos

Poner en práctica los conocimientos de: POO, memoria dinámica y GUI.

3 Especificación de requerimientos

Para la entrega se establecen los siguientes requerimientos como parte fundamental dentro del desarrollo del videojuego:

- Físicas: Incluir al menos tres modelos físicos (Movimiento rectilíneo no cuenta).
- POO y memoria dinámica de uso obligatorio.
- Al menos una herencia propia.
- La noción de dificultad al jugar debe ser parte del desarrollo, debe mostrar evidencia del proceso de abstracción de la dificultad. El videojuego debe ser agradable, usable e interesante para el usuario final. Establezca la opción de configuración de dificultad para uno de los niveles (Esto no puede reducirse a soluciones triviales: recortar el tiempo o subir la cantidad de ítems a coleccionar, etc).

- Se debe implementar usando la interfaz gráfica de QT.

4 Metodología y planificación

Nuestro proyecto se desarrolló en 4 fases claves, esto nos permitió una distribución más equitativa de trabajo además de un flujo mucho más limpio y la posibilidad de retroalimentarnos mutuamente.

4.1 Fase 1: Físicas y lógica principal

Inicialmente, trabajamos de manera conjunta en la rama principal del repositorio con el fin de estandarizar los métodos y establecer la lógica base del juego. Aquí desarrollamos el motor físico, la entidad de juego, el personaje y sus atributos.

4.2 Fase 2: Desarrollo clases secundarias

Empezamos con el desarrollo de las distintas clases adicionales que serían de utilidad dentro de cada nivel, como las plataformas, la clase base nivel, los robots de seguridad.

4.3 Fase 3: Integración UI

Una vez establecimos los cimientos lógicos, empezamos a crear, modificar y configurar los respectivos sprites y escenarios para cada nivel. Existieron varias versiones de modelos para el personaje, así como variaciones en el escenario.

4.4 Fase 4: Creación de niveles

Cuando ya establecimos nuestros sprites y el escenario, comenzamos el desarrollo de cada nivel, configurando a detalle cada elemento dentro del escenario. Se inicia con la aplicación de las físicas, la lógica del nivel, el posicionamiento de los distintos objetos, la música y efectos de sonido.

5 Diseño y arquitectura del sistema

El diseño y arquitectura del sistema se concibió implementando un enfoque modular, permitiendo separar estrictamente la capa lógica de la capa de interfaz gráfica (GUI). Esta separación de responsabilidades garantiza un bajo acoplamiento entre los componentes, facilitando el mantenimiento y la escalabilidad del software.

A continuación, se presenta el diagrama de clases resumido que ilustra la jerarquía y las interacciones fundamentales entre los componentes del videojuego:

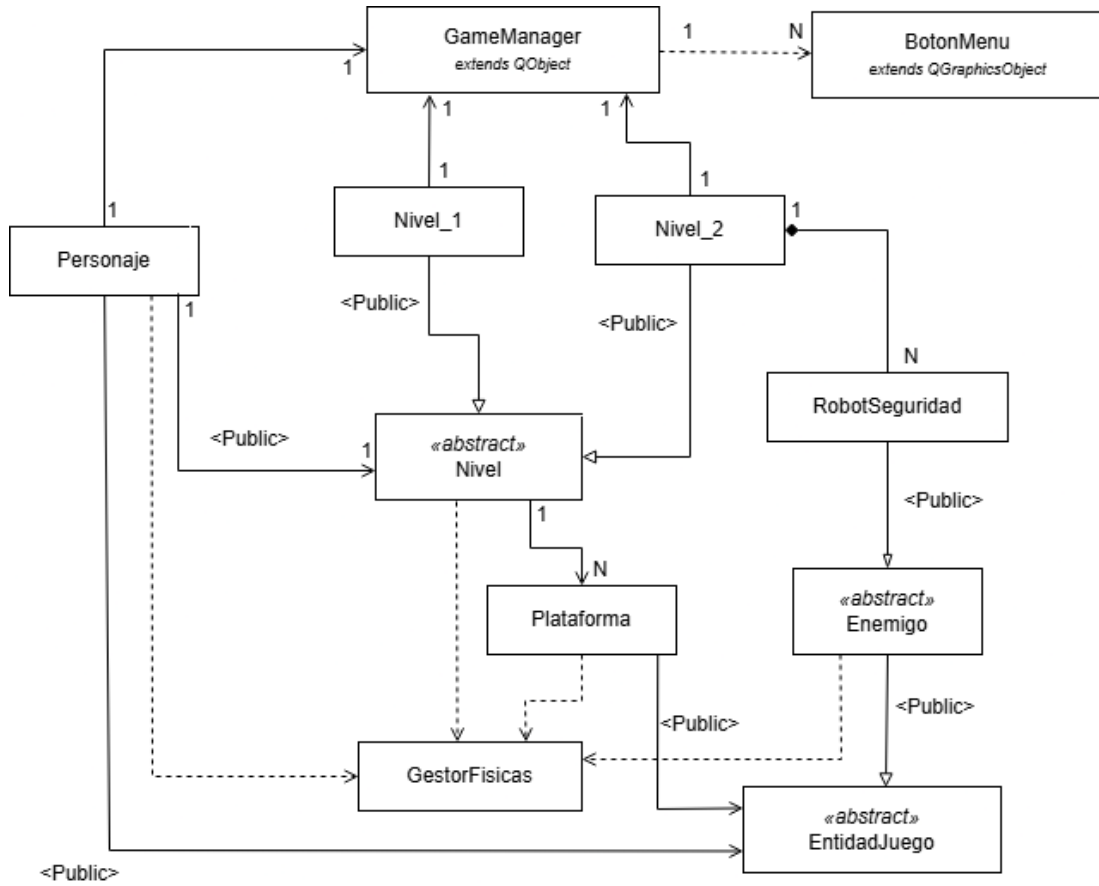


Figure 1: Diagrama de clases resumido enfocado en la separación de capas.

Para lograr este aislamiento, se implementó una clase controladora maestra llamada **GameManager**. Este componente se encarga de gestionar de manera centralizada todo lo relacionado con las visuales que observa el usuario, actuando como el núcleo de la interfaz gráfica y puente de comunicación hacia la lógica interna.

5.1 Capa lógica

La capa lógica está compuesta por las clases **Nivel_1**, **Nivel_2**, **Personaje**, **Plataforma**, **GestorFisicas**, **RobotSeguridad** y sus respectivas clases base abstractas: **EntidadJuego**, **Enemigo** (clase derivada de **EntidadJuego**) y **Nivel**.

Estos elementos son responsables de procesar los estados y mecánicas internas que ocurren tras bambalinas en el videojuego. Sus funciones principales abarcan la detección y resolución de colisiones a través del **GestorFisicas**, la actualización de coordenadas y la gestión del comportamiento de las entidades. Cabe destacar que las clases de los niveles actúan como ensambladoras de la escena (*Scene Builders*), lo que significa que cargan los recursos y estructuran el entorno lógico antes de entregar el contenedor visual listo para su renderizado.

5.2 Capa gráfica

La clase `GameManager` controla de forma absoluta la interfaz visual y el lienzo de renderizado de Qt. Sus responsabilidades principales dentro de la arquitectura son:

- **Gestión del ciclo de vida:** Inicializar los subsistemas gráficos del juego y administrar las transiciones globales de la aplicación.
- **Administración del lienzo:** Controlar la ventana principal, el viewport de dibujo y el montaje dinámico de las escenas visuales (`QGraphicsScene`) que son devueltas por los niveles correspondientes.
- **Abstracción de eventos de hardware:** Capturar centralizadamente los eventos de entrada del teclado y del mouse utilizando el sistema de eventos nativo de Qt, evitando que las clases lógicas tengan que comunicarse de forma directa con los periféricos físicos.
- **Máquina de estados de flujo:** Orquestrar los cambios de estado del juego, regulando la alternancia entre el Menú Principal, el `Nivel_1` y el `Nivel_2`.
- **Sincronización mediante GameTick:** Controlar el ciclo de actualización visual constante mediante un temporizador (`QTimer`), cuyos pulsos discretos o *Game Ticks* notifican periódicamente a la capa lógica del nivel activo para refrescar su estado antes de dibujar el siguiente fotograma.

Asimismo, la clase `BotonMenu` se integra en esta capa visual como un componente interactivo reutilizable. Su único propósito es encapsular la lógica de construcción, posicionamiento estético y captura de clicks de los elementos del menú dentro de las escenas de Qt, logrando la reutilización de código y manteniendo limpia la declaración de la interfaz de usuario.

6 Desarrollo e implementación

El funcionamiento interno del videojuego se articula mediante un ciclo de ejecución asíncrono controlado y la distribución de responsabilidades en métodos críticos de actualización. A continuación, se describen las decisiones técnicas, la lógica interna y los modelos algorítmicos implementados para los componentes más importantes del sistema.

6.1 Arquitectura del Bucle de Juego y Manejo del Tiempo (Δt)

Una de las decisiones técnicas importantes en el desarrollo fue desvincular la velocidad de las físicas y animaciones de la tasa de refresco de fotogramas (*frame rate*) del monitor. Para lograr un movimiento homogéneo e independiente del hardware, se implementó el uso de un diferencial de tiempo o **Delta Time** (Δt), el cual es calculado por el `GameManager` y propagado hacia los siguientes métodos en cada ciclo del reloj:

- `actualizarNivel(double dt)`: Este método centraliza la actualización del entorno. Recibe el Δt para sincronizar temporalmente el avance de la lógica global, invocar

la evaluación de colisiones dinámicas y, específicamente en el escenario del Nivel 2, disparar los hilos de comprobación del rango de alerta del Agente Inteligente para determinar si el jugador ha sido descubierto.

- **actualizarJugadorNivel(double dt):** Encargado exclusivamente de la entidad del personaje principal. Utiliza el diferencial de tiempo para calcular ecuaciones cinemáticas elementales (posición, velocidad y aceleración por gravedad), asegurando que el desplazamiento sea suave. Simultáneamente, gestiona las transiciones de la hoja de *sprites* (*spritesheet*) para actualizar las animaciones de correr, saltar o recibir daño de acuerdo al estado del juego.

6.2 Algoritmia del Agente Inteligente (RobotSeguridad)

El RobotSeguridad presente en el Nivel 2 opera bajo el paradigma de un Agente Inteligente Autónomo. Su lógica interna no es lineal, sino que implementa una **Máquina de Estados Finitos (FSM)** estructurada bajo el ciclo clásico de control de agentes: *Percibir*, *Razonar* y *Actuar*.

El método `Robot::tick(double dt)` actúa como el despachador de este ciclo y evalúa de forma discreta la secuencia de comportamiento del autómata:

1. **Percibir** (`percibir()`): Implementa un algoritmo de detección espacial (generalmente basado en cálculo de distancias euclidianas o proyección de vectores de visión). Si las coordenadas del personaje entran dentro del radio delimitado de alerta del robot, el entorno se considera hostil y se gatilla un cambio inmediato en las variables de estado del agente.
2. **Razonar** (`razonar()`): Evalúa la situación actual con base en su máquina de estados. Si el estado es de *Patrullaje*, calcula rutas predefinidas. Si el estado cambia a *Persecución* (tras haber detectado al jugador), el algoritmo toma decisiones dinámicas alterando los vectores de dirección para priorizar la aproximación al objetivo y recalcular la evasión inmediata de obstáculos estáticos.
3. **Actuar** (`actuar()`): Traduce las directrices de la fase de razonamiento en respuestas físicas. Este método aplica las fuerzas cinemáticas necesarias, altera la velocidad lineal del robot en el eje de coordenadas y ejecuta los algoritmos de `moverhaciaConEvacuacion` según el mapa de obstáculos circundantes.

6.3 Subsistema de Colisiones (GestorFisicas)

La resolución de colisiones es el algoritmo de mayor coste computacional dentro del bucle del juego, ejecutándose de manera omnipresente en cada *Game Tick*. Como decisión técnica de diseño para optimizar el rendimiento y evitar cálculos de geometría complejos en tiempo real, se implementó un algoritmo de ****Cajas de Colisión Alineadas con los Ejes**** (AABB, *Axis-Aligned Bounding Boxes*).

El **GestorFisicas** interviene en la actualización constante interceptando los rectángulos delimitadores de las entidades antes de consolidar su nueva posición en el plano. Si el algoritmo detecta una superposición de áreas en el eje coordenado (x, y) , detiene el vector

de velocidad de la entidad (como el **Personaje** colisionando con una **Plataforma**), anulando el desplazamiento e impidiendo el traspaso físico de texturas en la pantalla.

7 Resultados y discusión

7.1 Funcionamiento logrado

Se logró la implementación exitosa de dos niveles interactivos completamente funcionales dentro del videojuego. Ambos escenarios cuentan con un sistema operativo de selección de dificultad desde la interfaz del menú. El sistema responde correctamente al ciclo de actualización por *ticks* de tiempo, la renderización de las hojas de *sprites*, la resolución de colisiones en tiempo real y el comportamiento del autómata de seguridad en el segundo nivel.

7.2 Limitaciones actuales

A pesar de contar con el selector de dificultad integrado, actualmente solo el **Nivel_1** presenta una variación perceptible y balanceada en su jugabilidad al alterar sus parámetros. En el caso del **Nivel_2**, debido a la naturaleza y el diseño dinámico de su lógica interna (enfocada en el sigilo y la evasión del Agente Inteligente), las modificaciones de las variables de dificultad no producen un impacto lo suficientemente notable en la experiencia del usuario, quedando acotada a cambios sutiles en los vectores de velocidad.

7.3 Dificultades enfrentadas

El mayor desafío técnico durante el desarrollo radicó en la curva de aprendizaje asociada al ecosistema nativo de Qt. Específicamente, se presentó una complejidad alta al comprender y asimilar la arquitectura de gestión de memoria dinámica del *framework*, la cual difiere del C++ estándar debido al sistema de propiedad padre-hijo (*Object Trees*) de las clases derivadas de **QObject**. Controlar los ciclos de vida y la correcta liberación de recursos de memoria en punteros de clases como **QGraphicsScene**, **QKeyEvent** y la carga de texturas con **QPixmap** requirió de un proceso profundo de depuración para evitar fugas de memoria (*memory leaks*) y cierres inesperados de la aplicación.

7.4 Mejoras planeadas

Con el fin de escalar y hacer mejor el videojuego , pensando en un rehusa a futuro, se plantea las siguientes líneas de mejora:

- **Densidad de enemigos:** Diseñar e incorporar un arreglo de entidades enemigas activas dentro del **Nivel_1** para incrementar la interactividad del escenario inicial.
- **Expansión del conjunto de movimientos:** Enriquecer el árbol de mecánicas del personaje principal, permitiéndole ejecutar acciones de combate ofensivas (ataques de corto alcance) y defensivas, mitigando la dependencia exclusiva de las mecánicas de evasión de plataformas.

- **Refactorización de la IA:** Optimizar los estados de la máquina de estados del robot de seguridad para calibrar de forma más agresiva su velocidad y rango de visión de acuerdo con la dificultad seleccionada por el jugador.

8 Conclusiones

Para el desarrollo de este videojuego, contar con conocimientos previos sobre la gestión de memoria dinámica fue increíblemente útil, convirtiéndose en la base fundamental para poder estructurar todo el proyecto. De igual manera, el decidir un plan de desarrollo claro y diseñar el diagrama de clases desde el inicio impulsó notablemente el orden y la fluidez al momento de programar.

Durante todo el proceso nos enfrentamos a un sinfín de problemas técnicos, pero la persistencia nos permitió superarlos cada uno de ellos. La gran reflexión que nos deja este desarrollo es que la planificación inicial y la capacidad para afrontar y solucionar errores sobre la marcha son la verdadera llave para poder llevar a cabo con éxito un proyecto de esta magnitud.

Como aprendizaje principal, nos llevamos una experiencia real sobre el uso responsable de la memoria dinámica y el control de los apuntadores en C++. Asimismo, aprendimos a utilizar la interfaz gráfica de Qt, comprendiendo a fondo tanto las ventajas que ofrece para agilizar el diseño visual como los retos y limitaciones que conlleva acoplar este tipo de herramientas con la lógica pura del juego.

9 Referencias

A continuación, se detallan los enlaces a la documentación oficial y recursos utilizados para el desarrollo del sistema:

- **Gestión de ciclo de vida y eventos:**
Documentación oficial de QObject:
<https://doc.qt.io/qt-6/qobject.html>
- **Manejo de objetos gráficos y texturas:**
Documentación oficial de QPixmap:
<https://doc.qt.io/qt-6/qpixmap.html>
Documentación oficial de QGraphicsPixmapItem:
<https://doc.qt.io/qt-6/qgraphicspixmapitem.html>
- **Estructuras de datos y contenedores estándar:**
Guía de uso de `std::vector` (W3Schools):
https://www.w3schools.com/cpp/cpp_vectors.asp
Guía de uso de `std::list` (W3Schools):
https://www.w3schools.com/cpp/cpp_list.asp
- **Subsistema de audio y efectos de sonido:**
Documentación oficial de QMediaPlayer:
<https://doc.qt.io/qt-6/qmediaplayer.html>

Documentación oficial de QSoundEffect:
<https://doc.qt.io/qt-6/qsoundeffect.html>